

Data Management for Data science Project Proposal

By Nikhil Aditya Vasantavada

Project Proposal: Pharmacy Database Management System using Python and SQL

Team members : Nikhil Aditya Vasantavada

Problem Statement

The project aims to explore the limitations of traditional pharmacy management processes, which often rely on manual record-keeping or fragmented software that does not fully integrate inventory, prescriptions, billing, and patient information in one unified system. By designing a database-driven pharmacy management application using Python and SQL, we will try to improve accuracy in medicine dispensing, reduce stock-out/overstock situations, and streamline day-to-day pharmacy operations. This system will focus on managing medicines, suppliers, customers, and prescriptions in a structured way, making data retrieval and updates more efficient and reliable.

Strategy

The main strategy will be to build a centralized relational database that stores all critical pharmacy data such as drugs, categories, stock levels, prescriptions, and sales transactions, and connect it to a Python-based application for user interaction. These enhancements align with concepts in database management and application development, along with topics we've covered in class, such as ER modeling, normalization, SQL queries, and basic Create, Read, Update, Delete -based application design. Furthermore, integrating Python for the front-end logic with SQL for the back-end storage gives us a comprehensive solution that bridges both programming and database principles.

Importance

This project is crucial, as it addresses gaps in current small to mid-sized pharmacy workflows, where errors in stock management or prescription handling can directly affect patient safety and business performance. In the pharmacy domain, accuracy, timely availability of medicines, and proper record management are closely tied to regulatory compliance and customer trust. The ability to maintain up-to-date records of inventory, prescriptions, and sales could make operations smoother and help the pharmacy make data-driven decisions, such as identifying fast-moving drugs or predicting when to reorder stock.

Issues and Similar Works

Traditional pharmacy management techniques usually lack real-time, integrated digital records, which limits their ability to track stock movements, prescription histories, and expiry dates in a reliable way. Existing pharmacy software solutions often focus on either inventory or billing but may not provide a customizable, learning-focused environment suitable for academic projects and experimentation. Recent works on pharmacy management systems emphasize features such as automated stock alerts, expiry tracking, and patient history, but many commercial tools are closed-source and cannot be easily extended using Python and SQL by students.

Data Collection

The primary data sources will include:

- **Transaction Data:** Details of each medicine sale, including date, time, medicine ID, quantity, price, and payment method.
- **Inventory Data:** Records of all medicines (name, batch number, manufacturer, expiry date, quantity in stock, purchase price, selling price, and supplier details).
- **Prescription Data:** Simulated or sample prescriptions containing patient details, doctor details, prescribed medicines, dosage, and duration.
- **Master Data:** Static or semi-static data such as medicine categories, suppliers, and user roles (pharmacist, admin).

Data Cleaning and Visualization

The collected pharmacy data will be cleaned to improve its quality and reliability. Data cleaning will include removing duplicate records, handling missing values in inventory fields and correcting inconsistent formats such as name, dates etc. Ensuring that the data set is accurate and suitable for further analysis, after cleaning the data visualization techniques will be used to better understand pharmacy operations such as daily or monthly medicine sales for example medicines with near expiry dates, frequently sold medicines. The techniques include python libraries like Matplotlib which will help in clear insights and support decision for inventory management.

Data Storage

Data will be stored in a SQL database. Tables such as Medicines, Suppliers, Customers/Patients, Prescriptions, Prescription_Items, Sales, and Users will be created to store structured data in a normalized form. Relationships will be defined using primary and foreign keys (for example, Prescription_Items linked to Prescriptions and Medicines) to ensure referential integrity and support efficient queries for reporting and daily operations.

Methodology

To build a responsive pharmacy management system, we will use:

- **Relational Database Design:** ER diagrams and normalization to design consistent tables for medicines, prescriptions, suppliers, and sales.
- **Create, Read, Update, Delete Operations with Python:** Python to implement create, read, update, and delete operations on the SQL database through a simple GUI.
- **Basic Business Logic:** Rules to validate stock availability before processing a sale, update inventory after each transaction, and flag medicines approaching expiry based on their dates.

Data Collection and Preprocessing

- **Data Entry and Import:** Create initial datasets of medicines, suppliers, and sample patients/prescriptions, either manually or via CSV import, and load them into the SQL database.
- **Database Setup:** Design and implement schemas for the SQL database, creating tables, constraints, and indexes to hold structured pharmacy data and optimize common queries (e.g., search by medicine name or expiry date).
- **Application Development:** Build a Python-based interface that allows pharmacists to add medicines, record sales, manage prescriptions, and view reports, all interacting with the SQL database in the backend.
- **Feature Engineering and Integration:** Generate useful features such as low-stock indicators, near-expiry lists, daily/weekly sales summaries, and top-selling medicines to improve decision support.
- **Evaluation and Tuning:** Evaluate the system in terms of query performance, data consistency, and usability, refining database indexing, constraints, and Python-side input validation as needed.
- **Testing:** Perform scenario-based testing, such as selling out-of-stock items, entering invalid prescription IDs, or handling expired medicines, to verify that the system enforces business rules correctly.

Machine learning

Machine learning techniques will be applied to extract useful predictions from pharmacy data. By training machine learning models such as linear regression or random forest, this system can estimate future demand for different medicines. This would help the pharmacy avoid stock shortages or over stock situations.

The machine learning model will be trained on transaction data and evaluated using performance metrics like prediction of accuracy or error rate.

Integrating machine learning into this system will enhance the pharmacy management platform by enabling data driven forecasting and smarter inventory planning.

System Accuracy and Reliability

Using consistency checks and validation rules, we will assess how well the system prevents invalid operations (such as negative stock or selling expired medicines). We will also verify accuracy by comparing manual stock calculations with automatically computed stock levels after a sequence of simulated transactions.

User and Workflow Efficiency

Here I will observe how many steps are required to perform common tasks such as adding new medicines, processing a prescription, or checking stock, and aim to minimize redundant actions. By creating a more organized and responsive pharmacy database management system, this project will showcase the power of database-backed applications in improving operational efficiency and data reliability in real-world domains like healthcare and retail pharmacy.